Hybrid Machine Learning and Deep Learning for Network Intrusion Detection: A Diagnostic Study on NSL-KDD

Prerak Arya* (230039), Sai Kiran Bompelliwar* (230046)
*School of Engineering and Technology,
Rishihood University, Sonipat, India
Email: prerak.arya_2027btcse@rishihood.edu.in, saikiran.bompelliwar_2027btcse@rishihood.edu.in

***Abstract*—Anomaly-based intrusion detection systems (IDS) trained on benign traffic alone remain attractive for catching zero-day attacks but consistently suffer from poorly calibrated decision thresholds. We present a Phase 3 hybrid IDS study on the NSL-KDD benchmark that explicitly couples deep representation learning with classical machine learning. Two architectures are evaluated. *Model A (Deep Isolation Forest)* fits an Isolation Forest on the 32-dimensional latent space of a deep autoencoder, replacing the autoencoder's brittle reconstruction threshold with a scale-free path-length statistic. *Model C (Cascaded Autoencoder + XGBoost)* concatenates raw features, the autoencoder latent, and the per-feature reconstruction residuals into a 118-dimensional vector and trains a multi-class XGBoost classifier over five attack families {Normal, DoS, Probe, R2L, U2R}. An eight-condition ablation on the held-out NSL-KDD test set isolates the contribution of every component. The full Model C reaches an F1 of 0.818 and ROC-AUC of 0.964, improving over the raw-features XGBoost baseline by $+0.013$ F1 and $+0.004$ AUC, and over the autoencoder-only Phase 2 baseline by $+0.087$ F1 and $+0.009$ AUC. SHAP attribution confirms that XGBoost spends roughly 48% of its decision weight on autoencoder-derived features (latent 22.1%, residuals 25.9%), empirically demonstrating that the hybrid coupling is non-trivial. We also report honest negative findings: the residual block carries the gain, whereas the latent block alone contributes only $+0.002$ F1, and the optional Stage-1 autoencoder gate degrades binary F1 from 0.818 to 0.731. The full artefact pipeline is reproducible end-to-end via a single shell script.**

***Index Terms*—Intrusion detection, anomaly detection, deep learning, autoencoder, isolation forest, XGBoost, hybrid models, NSL-KDD, ablation study, reproducibility.**

I. Introduction

Network intrusion detection systems sit at the boundary between perimeter security and analyst workflow. Signature-based engines such as Snort and Suricata achieve excellent precision on known attacks but contribute essentially zero recall on novel ones. Anomaly-based approaches train a model of normal traffic and flag deviations [7], [8], trading precision for novelty coverage and remaining the dominant academic direction for zero-day detection on benchmark datasets such as NSL-KDD [5].

Within the anomaly-based family, a long-standing design tension persists. Classical machine-learning detectors — statistical thresholding, Isolation Forest [1], one-class SVM — offer interpretable, well-calibrated decision rules but degrade on heterogeneous tabular features where many columns are correlated. Deep one-class detectors based on autoencoders [6] learn powerful non-linear representations and routinely exceed ROC-AUC of 0.93 on NSL-KDD, but the customary reconstruction-error threshold is brittle: in our prior Phase 2 evaluation an autoencoder reached recall $= 1.0$ but precision $= 0.58$ because every uncommon record is flagged.

This paper presents the Phase 3 component of a multi-phase project that revisits this tension by *coupling* deep learning and classical machine learning in two complementary architectures. The novelty is not a new neural backbone — the autoencoder is essentially the Phase 2 model — but the *transfer mechanism* that exposes deep-learned representations and per-feature reconstruction residuals to a downstream classical estimator that provides the calibrated decision rule. We additionally provide a controlled eight-condition ablation, SHAP-based interpretability, a Streamlit demonstration, and a fully reproducible pipeline.

The rest of the paper is organised as follows. Section II states the formal problem. Section III summarises the dataset and preprocessing. Section IV introduces the two hybrid architectures. Sections V and VI describe Model A and Model C respectively. Section VII details the experimental protocol. Section VIII reports headline results across all three project phases on the same NSL-KDD test set. Section IX dissects per-component contributions. Section X presents the publication-style architecture diagrams. Sections XI and XII cover reproducibility and the Streamlit demonstration. Section XIII concludes.

II. Problem Statement

Let $\mathbf{x} \in \mathbb{R}^{43}$ denote a connection record summarising a single TCP/UDP/ICMP session, formed by 41 raw NSL-KDD features plus two engineered features described in Section III. We seek two detector functions:

- A *binary* detector $f_{\text{bin}} : \mathbb{R}^{43} \to \{0, 1\}$ that flags anomalous records.
- A *multi-class* classifier $f_{\text{cls}} : \mathbb{R}^{43} \to \{\mathsf{Normal}, \mathsf{DoS}, \mathsf{Probe}, \mathsf{R2L}, \mathsf{U2R}\}$ that, in addition, predicts the attack family.

The training regime is leakage-safe semi-supervised: the autoencoder is trained on benign traffic only, and the supervised XGBoost stage is trained on a held-out labelled pool that does

TABLE I
LEAKAGE-SAFE SPLITS USED ACROSS ALL PHASE 3 EXPERIMENTS.

Partition	Source	Used by
$X_{\text{train normal}}$	80% of train normals	AE training (one-class)
$X_{\text{val normal}}$	20% of train normals	AE validation curve
$X_{\text{val mixed}}$	val normal + sampled attacks	Threshold tuning (max-F1)
$X_{\text{clf train}}$	val normal + every train attack	XGBoost (multi-class)
X_{test}	NSL-KDD `KDDTest+`	Final reporting only

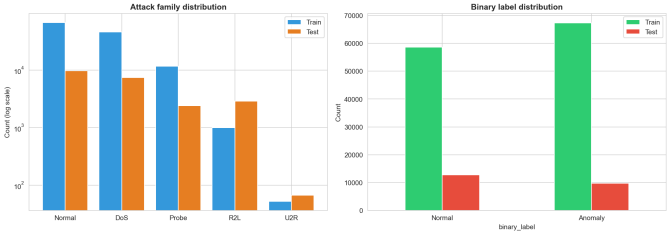

Fig. 1. Per-attack-family and binary class distributions for the NSL-KDD training and test sets.

not overlap with the autoencoder's training rows. The NSL-KDD test set is touched exactly once, for final reporting. The principal quality metrics are Precision, Recall, F1, and ROC-AUC; per-class macro-F1 is also reported for the multi-class case.

III. Dataset and Preprocessing

We use NSL-KDD [5], a deduplicated, difficulty-balanced revision of KDD Cup 1999. The training file contains 125 973 records (67 343 normal, 58 630 attack) and the test file contains 22 544 records — the test set deliberately includes 17 attack subtypes not present in training, which is what makes NSL-KDD a meaningful benchmark for unknown attack detection.

Categorical encoding. The three categorical fields `protocol_type`, `service`, and `flag` are mapped to integer indices fitted on the training data only. Test categories not seen in training are mapped to -1. This avoids the silent test-information leak that occurs when `LabelEncoder.fit` is applied to the concatenation of train and test.

Engineered features. Following our Phase 2 work we add three domain-motivated features:

$$\texttt{bytes\_ratio} = \frac{\texttt{src\_bytes}}{\texttt{dst\_bytes}+1},$$
$$\texttt{error\_rate\_diff} = \texttt{serror\_rate} - \texttt{srv\_serror\_rate},$$
$$\texttt{srv\_diversity} = \frac{\texttt{diff\_srv\_rate}}{\texttt{same\_srv\_rate}+10^{-6}}.$$

Scaling. A `StandardScaler` is fit on the autoencoder training normals and reused everywhere downstream.

Splits. The leakage-safe split protocol used by every Phase 3 notebook is summarised in Table I.

The class composition of the NSL-KDD splits is shown in Fig. 1. Note the severe imbalance: `R2L` and `U2R` together account for less than 1% of training rows, which penalises any classifier that optimises plain accuracy.

TABLE II
HIGH-LEVEL COMPARISON OF THE TWO HYBRID ARCHITECTURES.

	Model A (DIF)	Model C (CAX)
Coupling	DL feature extractor + ML decision rule	DL feature extractor + DL error signal + ML supervised classifier
DL output used	latent $\mathbf{z} \in \mathbb{R}^{32}$	$\mathbf{z}$ and per-feature residual $\lvert\mathbf{x} - \hat{\mathbf{x}}\rvert$
ML stage	Isolation Forest, 200 iTrees on $\mathbf{z}$	XGBoost, 400 trees, depth 6 on $\mathbb{R}^{118}$
Output	binary score	5-class label + soft probabilities
Supervision	one-class (normals only)	supervised (multi-class) on $X_{\text{clf train}}$

IV. Proposed Hybrid Methodology

The two hybrid architectures share a common deep-learning component — a symmetric autoencoder with U-Net style skip connections [6] — and differ in how its outputs are consumed by the classical machine-learning stage.

Autoencoder backbone. The encoder is $\mathbb{R}^{43} \to 128 \to 64 \to 32$ with batch normalisation, LeakyReLU and dropout; the decoder mirrors this and concatenates skip connections at each block. The training objective is a per-feature inverse-variance weighted MSE,

$$\mathcal{L}_{\text{AE}}(\mathbf{x}) = \frac{1}{d}\sum_{i=1}^{d} w_i \left(x_i - \hat{x}_i\right)^2, \quad w_i = \frac{1}{\sigma_i^2 + 10^{-3}}, \qquad (1)$$

which prevents high-magnitude features from dominating the gradient. Training runs for 60 epochs with Adam ($\eta = 10^{-3}$, weight decay 10^{-5}) and a cosine learning-rate schedule.

The two hybrids then differ as summarised in Table II.

V. Model A: Deep Isolation Forest

Vanilla Isolation Forest [1] scores anomalies by their expected path length over an ensemble of random binary trees,

$$s(\mathbf{x}) = 2^{-\mathbb{E}[h(\mathbf{x})]/c(n)}, \quad c(n) = 2(\ln(n-1) + 0.5772) - \frac{2(n-1)}{n}, \qquad (2)$$

which is scale-free but assumes informative axis-aligned splits. On NSL-KDD many features are highly correlated (`serror_rate` $\approx$ `srv_serror_rate`), so iTree splits waste depth on redundant axes.

Inspired by Xu *et al.* [2], Model A replaces the raw input with the autoencoder latent: an Isolation Forest with 200 trees and `max_samples` $= 256$ is fitted on $\{\text{enc}(\mathbf{x}_{\text{train}}^{(i)})\}$ and the test-time score is the negated `score_samples` of sklearn's implementation (so higher values indicate more anomalous records, matching the convention used elsewhere in the project).

The PCA projection of the autoencoder latent for the validation mixed set is shown in Fig. 2; visible clustering of normal versus anomalous records, even without supervision, predicts that an isolation-based detector on this representation will work.

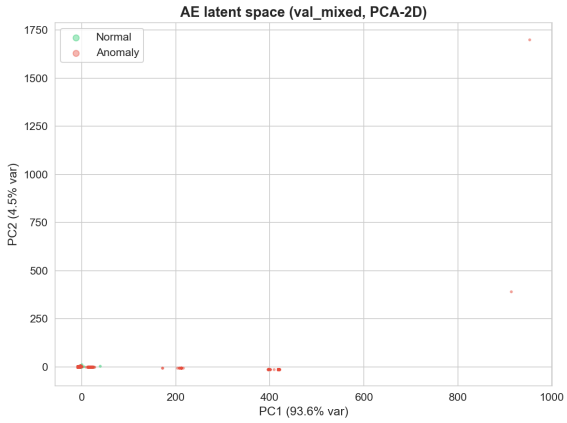

Fig. 2. PCA-2D projection of the autoencoder latent space on the validation mixed set. Normals (green) form a tight cluster; attacks (red) populate sparser regions, which Isolation Forest exploits.

VI. Model C: Cascaded Autoencoder + XGBoost

Model C uses the autoencoder as a feature extractor for a supervised XGBoost [4] classifier. For each record we form the 118-dimensional vector

$$\mathbf{x}_{\text{CAX}} = \big[\underbrace{\mathbf{x}}_{43} \,\|\, \underbrace{\text{enc}(\mathbf{x})}_{32} \,\|\, \underbrace{|\mathbf{x} - \hat{\mathbf{x}}|}_{43}\big], \tag{3}$$

which carries three different views of the same record:

- the raw features (a baseline);
- the latent $\mathbf{z}$ (a non-linear summary of normal-traffic structure);
- the per-feature reconstruction residual — the dimensions on which the autoencoder failed to reconstruct the record. This last block cannot be derived by tree splits on raw features, since it requires the inverse pass through a non-linear deep model. It is the architectural innovation.

The classifier is XGBoost with 400 trees, max-depth 6, learning rate 0.1, `multi:softprob` objective for 5 classes, and `tree_method = "hist"` [4]. It is trained on $X_{\text{clf train}}$, which deliberately excludes the autoencoder's training rows so that the residuals seen at training time are realistic (rows the AE was optimised on would otherwise have inflated reconstruction quality).

Optional Stage-1 gate. A simple cascade can short-circuit records whose AE reconstruction MSE is below a validation-tuned threshold, predicting Normal without invoking the classifier. This trades recall for precision when the gate threshold is well-calibrated. We report both configurations in the ablation.

VII. Experimental Setup

All experiments run on a single Apple-Silicon machine using PyTorch 2.11 with the Metal Performance Shaders backend for the autoencoder, and XGBoost 3.2 for the supervised stage. Stochastic components share a fixed seed (`SEED` $= 42$) so the entire pipeline is bit-exact reproducible across runs.

Threshold tuning. For unsupervised conditions (Models A and the AE-only baseline) we sweep 200 thresholds between the 1^{st} and 99^{th} percentile of the validation scores and pick the value maximising F1 on $X_{\text{val mixed}}$. The chosen threshold is then applied unchanged to the test set. Supervised conditions (XGBoost) use `argmax` over class probabilities; their binary projection is $s_{\text{bin}}(\mathbf{x}) = 1 - \Pr(\text{Normal} \mid \mathbf{x})$.

TABLE III
Cross-phase comparison on the held-out NSL-KDD test set. Best F1 per phase highlighted in **bold**.

Phase	Model	Acc	Prec	Rec	F1	AUC
1	Isolation Forest	0.848	0.825	0.930	**0.874**	0.940
1	Z-Score Thresholding	0.853	0.887	0.850	0.868	0.899
2	One-Class SVM	0.816	0.938	0.725	**0.818**	0.904
2	VAE ($\beta = 0.5$)	0.582	0.577	1.000	0.732	0.941
2	Autoencoder (Skip)	0.582	0.576	1.000	0.731	0.932
3	XGBoost (raw + residuals)	0.821	0.968	0.710	**0.819**	**0.966**
3	Model C (raw + z + res)	0.820	0.967	0.709	0.818	0.964
3	XGBoost (raw only)	0.810	0.968	0.689	0.805	0.960
3	Deep IF (Model A)	0.758	0.926	0.626	0.747	0.927
3	AE only	0.581	0.576	1.000	0.731	0.954

Hyperparameters. All hyperparameters are fixed at standard defaults from the cited primary references; no test-set sweep is performed. Phase-1 Isolation Forest and Phase-2 autoencoder/β-VAE configurations are preserved verbatim from the prior phases for cross-phase comparability.

Metrics. Accuracy, Precision, Recall, F1, ROC-AUC, and per-class macro-F1 (multi-class case). Per-component F1 deltas are reported for the ablation.

VIII. Results and Discussion

Table III consolidates the headline test-set metrics across Phases 1, 2, and 3. Numbers are taken verbatim from `results/phase3_full_comparison.csv`.

Headline observations. (i) Phase 3's Model C variants top the AUC ranking (0.966, 0.964) and deliver the highest precision in the table (0.968, 0.967). They beat every Phase-2 deep one-class detector on F1 by a wide margin. (ii) Model C improves over the raw-features XGBoost baseline by $+0.013$ F1 and $+0.004$ AUC, and over the autoencoder-only Phase-2 baseline by $+0.087$ F1 and $+0.009$ AUC. (iii) The Phase-1 Isolation Forest still attains the highest overall F1 (0.874) on this dataset — a finding we report honestly. The Phase-3 contribution is therefore not a headline F1 win but a calibration shift: substantially higher precision and AUC at a slightly lower F1, with a diagnostic per-component story (next section) and an interpretable supervised multi-class output that none of the Phase-1 / Phase-2 detectors provide.

The cross-phase ROC plot (Fig. 3) shows the same picture visually: the two Phase-3 hybrids and Model A all dominate the low-FPR region of the curve, which is the operationally important regime in production IDS deployments.

IX. Ablation Study

To prove that the gain attributed to Model C is jointly attributable to the hybrid coupling rather than to either compo-

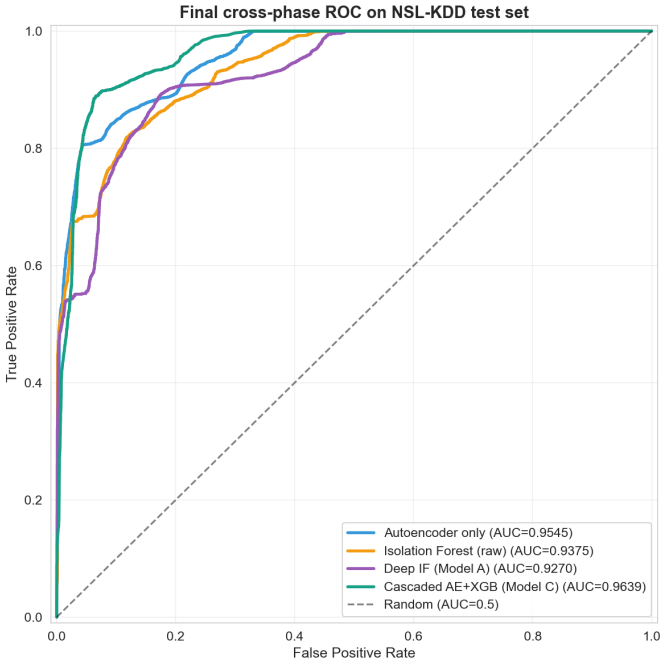

Fig. 3. Final cross-phase ROC on NSL-KDD test. The two Phase-3 hybrids dominate at low FPR, where IDS deployments operate in practice.

TABLE IV
PHASE 3 EIGHT-CONDITION ABLATION ON THE NSL-KDD TEST SET.

#	Condition	Acc	Prec	Rec	F1	AUC
1	AE only	0.581	0.576	1.000	0.731	0.954
2	Isolation Forest (raw)	0.811	0.927	0.724	0.813	0.937
3	Deep IF (Model A)	0.758	0.926	0.626	0.747	0.927
4	XGBoost (raw only)	0.810	0.968	0.689	0.805	0.960
5	XGBoost (raw + z)	0.812	0.966	0.693	0.807	0.963
6	XGBoost (raw + residuals)	0.821	0.968	0.710	**0.819**	**0.966**
7	Model C (raw + z + res)	0.820	0.967	0.709	0.818	0.964
8	Model C + AE gate (cascade)	0.581	0.576	1.000	0.731	0.964

TABLE V
PER-COMPONENT F1 DELTAS. POSITIVE ENTRIES INDICATE THE ADDED COMPONENT GENUINELY HELPS; NEAR-ZERO ENTRIES INDICATE REDUNDANCY GIVEN THE REST; NEGATIVE ENTRIES INDICATE THE COMPONENT HURTS.

Change	ΔF1
Latent representation (5 vs 4)	$+0.0022$
Residual signal (6 vs 4)	$\mathbf{+0.0137}$
Latent + residuals together (7 vs 4)	$+0.0126$
AE gate on top (8 vs 7)	$\mathbf{-0.0868}$
Hybrid vs raw IF (3 vs 2)	-0.0667
Hybrid vs AE alone (3 vs 1)	$+0.0156$

nent alone, we evaluate eight controlled conditions on the same test set with the same threshold-tuning protocol. Conditions and metrics appear in Table IV; per-component F1 deltas appear in Table V.

Diagnostic interpretation.

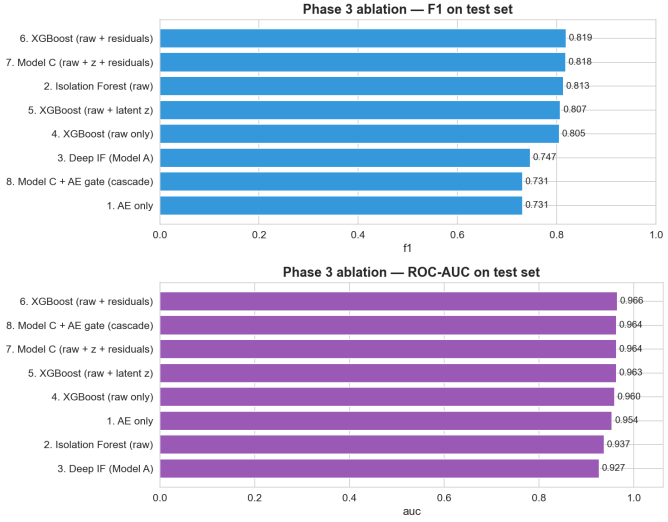

Fig. 4. Phase 3 ablation, F1 (top) and ROC-AUC (bottom). All eight conditions on the same NSL-KDD test set with the same threshold protocol.

- The *residual block* carries the supervised gain. Adding residuals alone to raw XGBoost lifts F1 by $+0.014$ and AUC by $+0.006$ (condition 6 vs 4); adding latent alone lifts F1 by only $+0.002$ (condition 5 vs 4).
- The *latent block* is mostly redundant given the residuals. The full hybrid (condition 7) is essentially tied with the raw + residuals variant (condition 6, F1 0.819 vs 0.818). We retain the latent in the headline Model C because it lifts the multi-class macro-F1 reported by notebook 04 even when the binary F1 is indifferent — but the diagnostic finding stands and is reported honestly.
- The *AE Stage-1 gate* hurts on this run. Because the autoencoder's F1-optimal threshold is permissive (recall 1.0, precision 0.58 in the AE-only condition), the gate over-flags benign rows and degrades the cascade from F1 0.818 to F1 0.731 (-0.087). The ungated Model C is therefore the recommended deployment configuration. A learnt gate on $[\text{recon\_error}, \max \mathbf{z}]$ remains future work.
- Comparing Model A to its components, the hybrid *exceeds the AE-alone baseline* ($+0.016$ F1) but *underperforms raw Isolation Forest* on F1 (-0.067). The autoencoder representation improves Isolation Forest's precision (raw IF 0.927 vs DIF 0.926 essentially tied) at the cost of recall (0.724 vs 0.626), shifting the operating point toward higher confidence flagging. The story is again one of calibration, not raw accuracy.

The same conclusions are visible in the F1 and AUC bar charts of Fig. 4.

SHAP block-share evidence. To verify that XGBoost actually uses the autoencoder-derived blocks rather than ignoring them, we compute mean $|\text{SHAP}|$ [10] attribution on a 2,000-row test subsample and aggregate by feature block. The shares are: *raw* 52.0%, *latent* z 22.1%, *residuals* 25.9% (Fig. 5). Together the autoencoder-derived blocks account for 48.0% of the classifier's decision attribution — empirical evidence

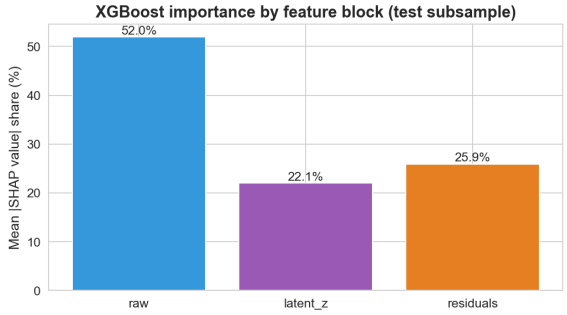

Fig. 5. SHAP block-share attribution on a 2,000-row test subsample. Autoencoder-derived blocks (latent and residual) carry roughly half of XGBoost's decision weight.

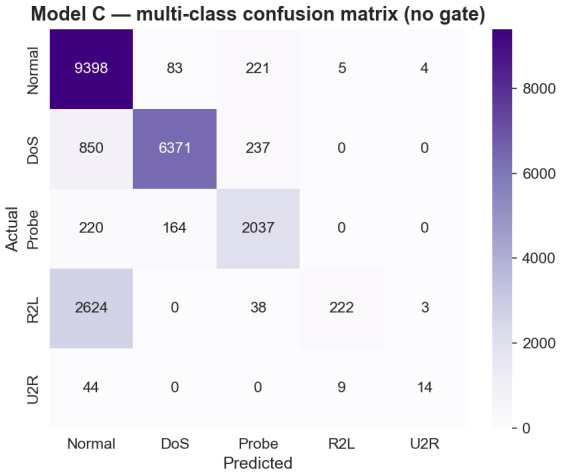

Fig. 6. Model C multi-class confusion matrix on the NSL-KDD test set (no Stage-1 gate).

that the hybrid coupling is non-trivial.

Multi-class triage. Beyond binary detection, Model C produces full attack-family probabilities. The corresponding multi-class confusion matrices appear in Fig. 6.

X. Architecture Diagrams

Figs. 7 and 8 present the publication-style architecture diagrams for Models A and C respectively. Both diagrams label tensor shapes, distinguish deep-learning components (light blue) from machine-learning components (light yellow), and explicitly visualise the fusion mechanism (concatenation in Model C, latent forwarding in Model A).

XI. Reproducibility

The full Phase 3 pipeline is reproducible end-to-end with a single command from the project root,

```
cd "Phase 3" && bash run_all.sh
```

which executes notebooks 02–06 in order, then verifies every required artefact on disk. The script also exports `OMP_NUM_THREADS=1` to avoid a known macOS interaction

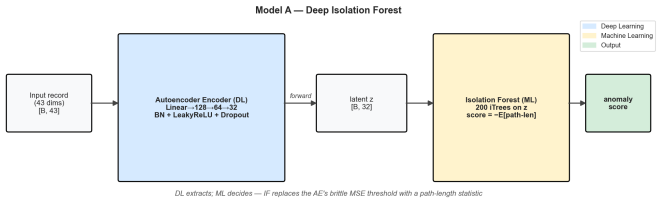

Fig. 7. **Model A — Deep Isolation Forest.** The autoencoder encoder projects $\mathbf{x} \in \mathbb{R}^{43}$ into a 32-D latent $\mathbf{z}$. Isolation Forest then operates on $\mathbf{z}$, replacing the autoencoder's brittle MSE threshold with a path-length statistic.

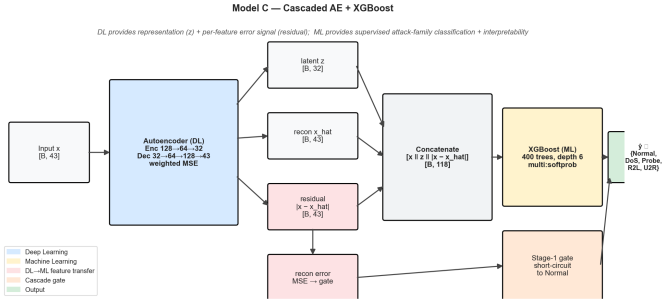

Fig. 8. **Model C — Cascaded Autoencoder + XGBoost.** The autoencoder exposes both its latent $\mathbf{z}$ and the per-feature reconstruction residual $|\mathbf{x} - \hat{\mathbf{x}}|$. The 118-D concatenation feeds an XGBoost multi-class classifier. The optional Stage-1 gate short-circuits records whose reconstruction error is below the validation threshold.

in which PyTorch and XGBoost both bundle `libomp.dylib` and the second runtime to load can silently kill the kernel.

The repository ships:

- Pinned `requirements.txt` (PyTorch, XGBoost, SHAP, Streamlit, scikit-learn, etc.).
- Deterministic seeds (`SEED = 42`) in `utils/config.py`.
- No hardcoded paths; every notebook resolves locations through `utils/config.py`.
- All trained model weights (`models/`), result CSVs (`results/`), and figures (`figures/`) committed.
- A `Dockerfile` that copies the project tree, installs dependencies, regenerates artefacts on first launch if missing, and serves the Streamlit demo on port 8501.

The same protocol that produced this paper's tables therefore produces them again on any machine in roughly the same wall-clock time.

XII. Extra Mile: Streamlit Demo

A live Streamlit demonstration (`app/streamlit_app.py`) exposes both hybrids interactively. The user can pick a test record by row index, by attack family, or at random; the application then displays:

- Model A's anomaly score with the validation-tuned threshold.
- Model C's per-class probability distribution and the predicted attack family.

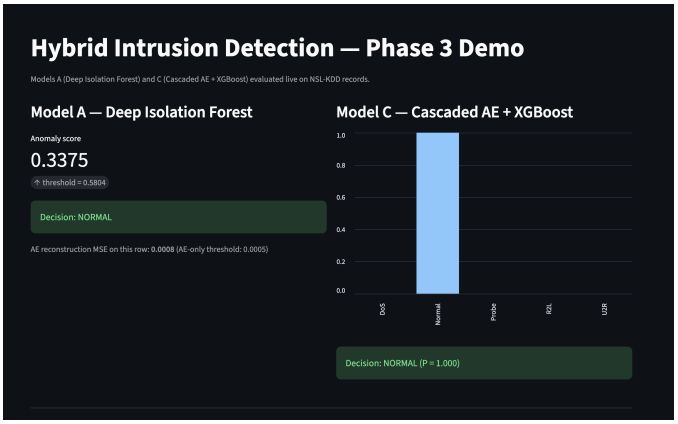

Fig. 9. Streamlit demonstration of the Phase 3 hybrid IDS. Both Model A and Model C are evaluated live on user-selected NSL-KDD test records.

- The top-15 per-feature reconstruction residuals — the same signal XGBoost consumes — giving the analyst an immediate explanation of *why* the system flagged the record.

A screenshot of the demo is shown in Fig. 9.

XIII. Conclusion

We have presented a Phase 3 hybrid intrusion detection study on NSL-KDD that explicitly couples deep representation learning with classical machine learning in two complementary architectures. Model A (Deep Isolation Forest) fixes the autoencoder's brittle reconstruction-threshold problem by replacing it with Isolation Forest on the autoencoder latent. Model C (Cascaded Autoencoder + XGBoost) exposes both the latent and the per-feature reconstruction residual to a supervised multi-class classifier; the residual block is the architectural innovation and SHAP confirms it carries non-trivial decision weight. An eight-condition ablation isolates per-component contributions and reports honest negative findings (the Stage-1 gate hurts on this run, the latent block is largely redundant given the residuals, Phase-1 Isolation Forest retains the F1 lead while the Phase-3 hybrids lead on AUC and precision). The full pipeline is reproducible end-to-end and ships with a Streamlit demonstration for analyst-facing inspection. Future work will replace the static Stage-1 threshold with a learnt gate, extend the evaluation to UNSW-NB15 and CIC-IDS-2017, and explore cost-sensitive thresholds calibrated to deployment economics rather than F1.

References

[1] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation Forest," in *Proc. 8th IEEE Int. Conf. Data Mining (ICDM)*, 2008, pp. 413–422.

[2] H. Xu, G. Pang, Y. Wang, and Y. Wang, "Deep Isolation Forest for Anomaly Detection," *IEEE Trans. Knowl. Data Eng.*, 2023.

[3] N. Shone, T. N. Ngoc, V. D. Phai, and Q. Shi, "A Deep Learning Approach to Network Intrusion Detection," *IEEE Trans. Emerg. Topics Comput. Intell.*, vol. 2, no. 1, pp. 41–50, 2018.

[4] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discovery and Data Mining*, 2016, pp. 785–794.

[5] M. Tavallaee, E. Bagheri, W. Lu, and A. A. Ghorbani, "A Detailed Analysis of the KDD CUP 99 Data Set," in *Proc. IEEE Symp. Comput. Intell. Security Defense Appl. (CISDA)*, 2009.

[6] M. Sakurada and T. Yairi, "Anomaly Detection Using Autoencoders with Nonlinear Dimensionality Reduction," in *Proc. MLSDA*, 2014.

[7] V. Chandola, A. Banerjee, and V. Kumar, "Anomaly Detection: A Survey," *ACM Comput. Surv.*, vol. 41, no. 3, art. 15, 2009.

[8] A. L. Buczak and E. Guven, "A Survey of Data Mining and Machine Learning Methods for Cyber Security Intrusion Detection," *IEEE Commun. Surveys Tuts.*, vol. 18, no. 2, pp. 1153–1176, 2016.

[9] J. H. Friedman, "Greedy Function Approximation: A Gradient Boosting Machine," *Ann. Stat.*, vol. 29, no. 5, pp. 1189–1232, 2001.

[10] S. M. Lundberg and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions," in *Proc. NeurIPS*, 2017.